DATA ANALYSIS FOR A LOAN CLASSIFICATION MODEL

Table of Contents

Introduction 2

Preprocessing 2

- **Discretization of continuous attributes** 2
- **Checking for missing values and imputing techniques** 3
- **Feature Engineering** 4
- **One Hot Encoding** 5
- **Normalization** 5

Initial Data Exploration 5

- **Apriori Algorithm for Association Rule Minning** 11
- **K-Medoid Clustering** 12

Learning the Classification Models 14

- **Decision Tree Classifier** 14
- **Naïve Bayes** 14
- **Random Tree Classifier** 14
- **Support Vector Machine** 15
- **Multiple Layer Perceptron Artificial Neaural Network** 15

Bias test Results 16

- **Reflection** 17

Conclusion 17

APENDIX 18

REFERENCES 25

Introduction

When data is provided to help consumers and businesses make great choices, The data itself is not always clear or clean to be used for this challenging tasks. Unclean data can lead to a lot of unwanted results and cause a lot of controversy among societies. Cleaning data and exploring it before using it on a model is important and it can help us understand patterns within the data to build a good model. In this paper data from a financial company has been supplied to build a model that predicts the loan failure tendency of a given application. Before constructing the first model, the provided data will be pre-processed and explored for any biases that could potentially affect the model's performance. These steps are important as Data cleaning involves identifying and correcting errors, inconsistencies, or missing values in the dataset to ensure its accuracy and reliability and data exploration helps us gain insights into patterns, relationships, and distributions within the dataset, enabling informed decisions during the modelling process. In addition to preprocessing, it is crucial to explore the data for any biases that could affect the model's predictions. Bias can arise from various sources, such as imbalanced data, underrepresentation of certain groups, or biased labelling. This problem is a classification problem which is dealing with predicting whether an applicant can pay a loan or not, so the final model will be built using classification algorithms and fine-tuned.

Preprocessing

Discretization of continuous attributes

Gama, J et al (1998) defined the technique of discretization as a process that involves breaking down continuous numerical values into a range of intervals that are equivalent to discrete categorical values. Since the dataset provided has financial information about individuals, the continuous values such as age, year of application, A-level scores, annual income, and total capital had to be categorically binned. These bins were split based on the attributes minimum, median, and maximum and with the categorical feature the bins were based on similarity. By doing this, it is easy to explore any bias in the data and understand each feature well based on its category bin.

Age Attribute discretization

The age attribute in the data set had continuous values with of ages from the minimum age being forty, the median age being forty-five and the maximum age being fifty. These ages can reveal some patterns in the dataset, so by making sure that they are correctly binned it will help spot out any potential bias against a certain age group. By using the binning technique, the ages were categorically split into groups of forties and fifties, respectively. This was done by looping through the age column and assigning each age group to its category in a new column.

Year Attribute discretization

The dataset covered application years from 1998 to 2023, enabling the establishment of interval buckets to sufficiently describe these years. The intervals were labelled as 'low years' (1998-2010),'median years' (2011-2019), 'COVID years' (2020-2021), and 'high years' (2022-2023). The 'COVID years' bucket was created specifically to evaluate the impact of the epidemic on loan applications. The years were binned into various categories using the panda’s library's "pd.cut" function, allowing for a granular investigation of temporal trends and the influence of exogenous events on application habits.

A-Level scores Attribute Discretization

The scores were also continuous values which needed to be discrete as each score can be unique or similar to others in this column. In order to address the issue of continuous values for the scores, the buckets decided to discretize this attribute into three categories was low scores, medium scores, and high scores. The range of scores varied from forty being the lowest to ninety being the highest, with sixty-five being the median score. By using conditional statements, the data was split successfully into these categories and assigning them to a new column. This was done by looping through the score column and assigning each interval set to a new score column.

Annual income Attribute Discretization

This attribute had to be discretized as one's annual income can continuously change, and multiple people can have the same income but different loan payment outcomes. The annual income in the data ranged from twenty-five thousand being the lowest, ninety-five thousand being the median income and hundred-fifty thousand being the highest annual income. The categories created were also low, high, and median income respectively to inspect any negative and positive bias based on income.

Total capital Attribute Discretization

The total capital for each business was also a continuous attribute with each business having different amounts of capital ranging from 20,000 as the lowest capital, 49,888 as the median, and 80,000 as the highest capital available in the business. Similar to the annual income attribute, the buckets for these attributes were created as lowest median and highest, respectively.

Checking for missing values and imputing techniques

The data supplied had a few features that had missing values in them, these values if not replaced properly they can influence negative bias and analysing the data will be difficult as the data doesn't have some values in it. Using the built-in python function is null and sum, it was easy to check for any missing values and count how many they are all together for each feature in the data. After checking for the missing values, the referee attributes were found to have the most number of missing values in the dataset. They are multiple ways to manage this missing data and one of them is by imputing the missing values. Nguyen, M. (2020) defined data imputation as a statistical process in which values that make sense are substituted for missing data. due to the nature of the values in the attributes it was best to use specific value imputation as the missing values in these attributes would mean that the reference was not available at the time of the application. The missing values were replaced using the python built in function "fillna" with the string "Not available" to show that the reference is not available rather than having it as Nan.

```
Gender                          0
Age at which Loan was Taken     0
Year of Application             0
A-levels                        0
Education                       0
Employment type                 0
Current Employment Sector       0
Proposed Business Sector        0
Current Annual Income           0
Total Capital                   0
Address                         0
Refree Present                  0
Refree 1                    17681
Refree 2                    17681
Refree 3                    17681
Loan Payment Failure            0
dtype: int64
```

Figure 1 Missing values in data

```
Gender                       0
Age at which Loan was Taken  0
Year of Application          0
A-levels                     0
Education                    0
Employment type              0
Current Employment Sector    0
Proposed Business Sector     0
Current Annual Income        0
Total Capital                0
Address                      0
Refree Present               0
Refree 1                     0
Refree 2                     0
Refree 3                     0
Loan Payment Failure         0
dtype: int64
```

Figure 2 Data after filling the Null values

Feature Engineering

The data set provided had features that had important values to contribute towards the process of predicting the loan failure payment. These features did not have any importance to the model unless their values were extracted and properly sorted, the features that needed engineering were the referee features from the first one to the third. The values in these features were in a tuple format with the key being the job type and the value was the salary, the job types were contractual, engineering and salaried. New features were created from these values to count how many jobs appeared in each reference at the time of the application being made. for the for the salaried job type, buckets of low, medium and high salaries were created to also inspect the count of each salary bucket present in the applicant's referee. By using a lambda function that took the job type and the referee columns as arguments, the function separated the tuple values and counted the sum of how many times a job type is appearing in the columns for each applicant. Similarly, to the salaried job type, the lambda function was used to create buckets of three and count how many times each

bucket appears for an application in each of the rows. the function separated the tuples and targeted the salary values only and then set intervals of the amounts to the lowest salary being forty-nine thousand, nine hundred ninety-nine, the median being fifty thousand to eighty-nine thousand, nine-hundred ninety-nine and the highest being from ninety thousand and above.

After the new features were introduced to the dataset, the dataset was edited by dropping the original dataset features and only keeping the features that have been bucketed and engineered. This ensured that the data to be used on the model is clean enough and has no values that will be rejected by any algorithm or further cleaning processes. These new features can help supply the model information that was not clearly put out to be used for the model, by manually extracting them the model's performance can be improved.

One Hot Encoding

After cleaning and engineering the dataset, a new dataset was created to hold these new features. The features in the new dataset could not be used on a model as it had numbers and strings which may confuse the model to prioritise unimportant values when not needed. The dataset was the encoded using one hot encoding which converts all the categorical data to ones and zeros and Boolean sometimes Boolean statements. For this dataset, this was archived by using the pandas built-in function called "get dummies" which created new Boolean columns for each of the categorical features. After this process the dataset was now ready to be used on different algorithms and models to get the best results during both exploration and model building.

Normalization

The encoded dataset now has values that are between one and zero, this still does not mean that the data is uniformly scaled. It is important to normalize the data as it ensures that no feature is dominating the others during the model learning process. For normalizing the data provided, standard scaler is used to scale out the data for better algorithm performance and training efficiency for the models that require scaled data. Raschka, S. (2015) defined standardization is a feature scaling procedure that gives the data we have the property of a standard normal distribution. Each feature's mean is cantered at 0 and the feature column has a standard deviation.

Initial Data Exploration

The data was examined using a number of methodologies to determine how certain characteristics influence loan payment failure outcomes. The first investigation focused on the impact of age groups, indicating that applicants in their forties had a higher rate of loan payment defaults than those in their fifties. This gap shows a potential bias against the younger age group, which could affect predictive model results unequally.

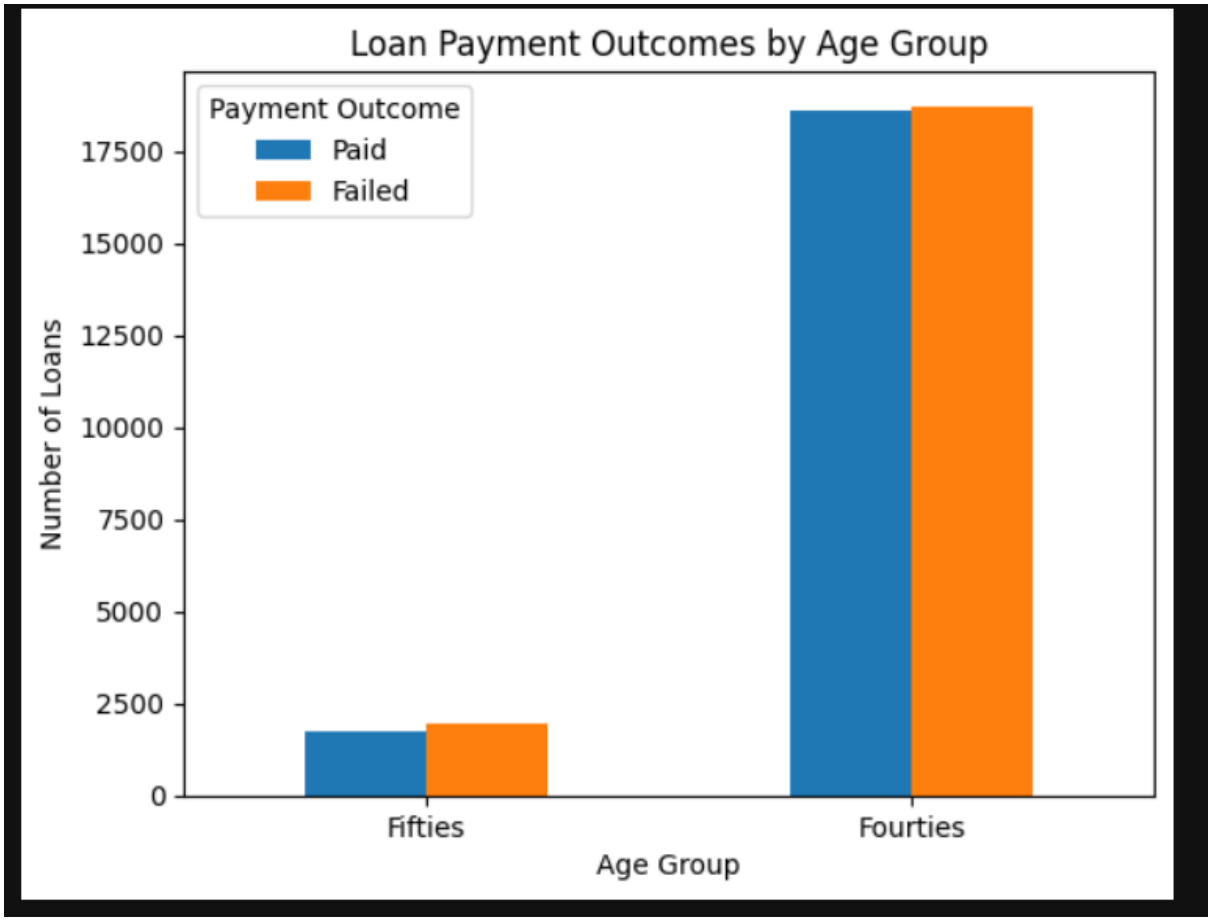

Figure 3 Bar graph for Loan payment outcomes by age

Further analysis was conducted on the data segmented by year brackets to determine their contribution to loan payment failures. The results, displayed in a bar graph, indicated that the years prior to 2010 experienced a roughly equal number of loan payment failures and successes. However, a significant increase in loan payment failures was observed during the COVID-19 years, implying that models trained on this data may be biased towards this particular time frame.

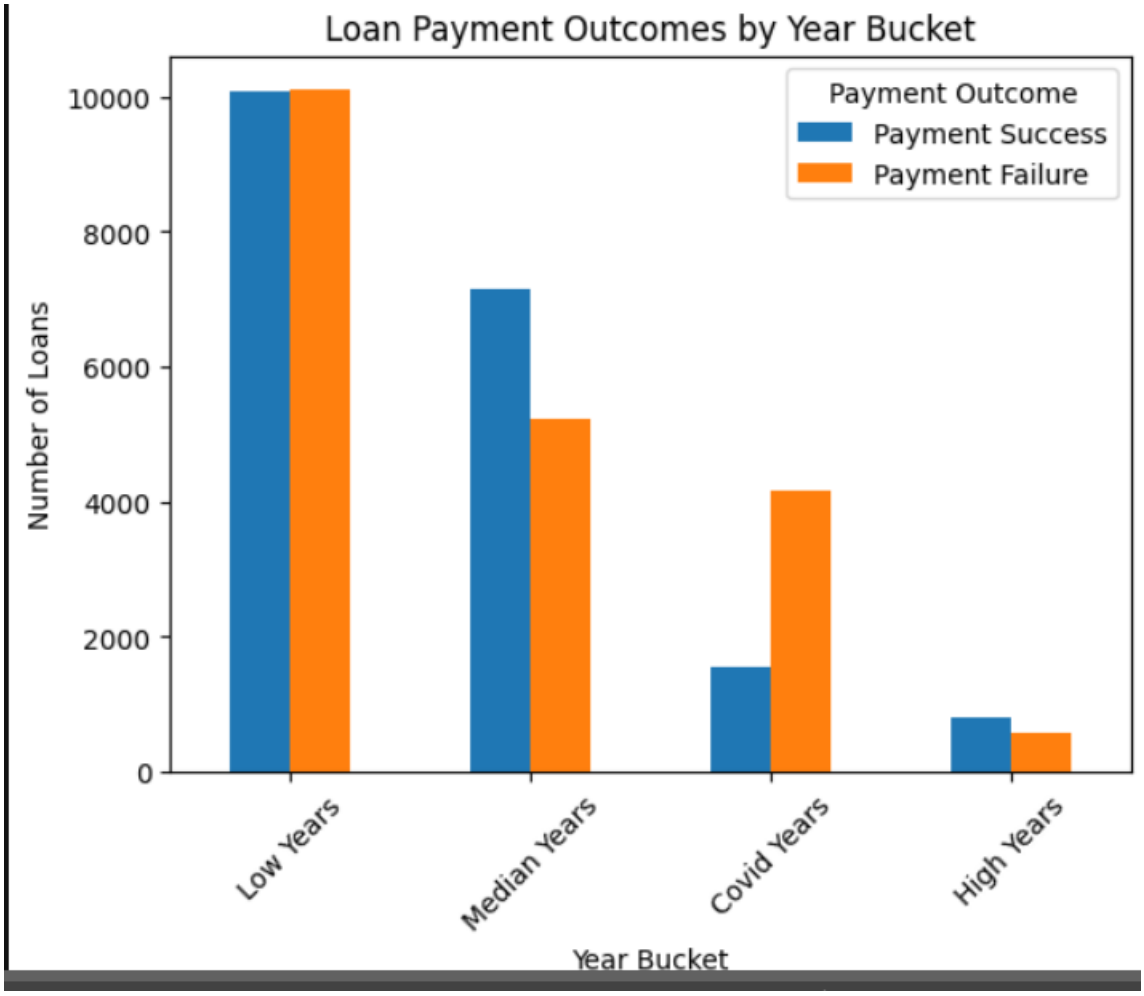

Figure 4 Loan payment outcomes by year bucket

The role of annual income was also examined to assess its impact on model performance. It was noted that all loan applications from the low-income group resulted in payment failures, with no recorded successes. This stark contrast may introduce an unethical bias against applicants with lower incomes.

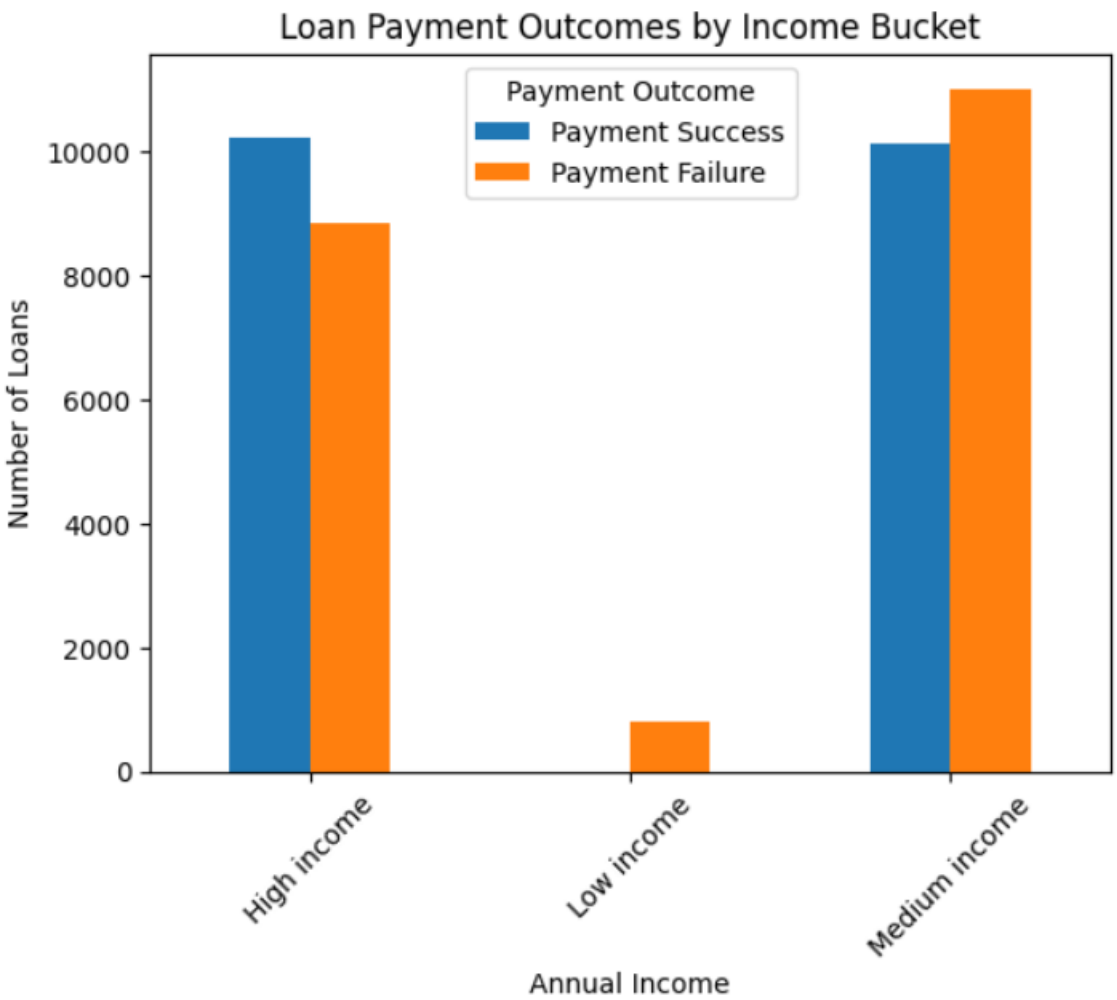

Figure 5 Loan Payment outcomes by Income Bucket

When analysing the capital associated with each loan application, it was found that the medium capital bracket had the highest number of loan payment failures. Interestingly, even applications with high capital were not immune to this trend, suggesting that this feature could lead to biased predictions.

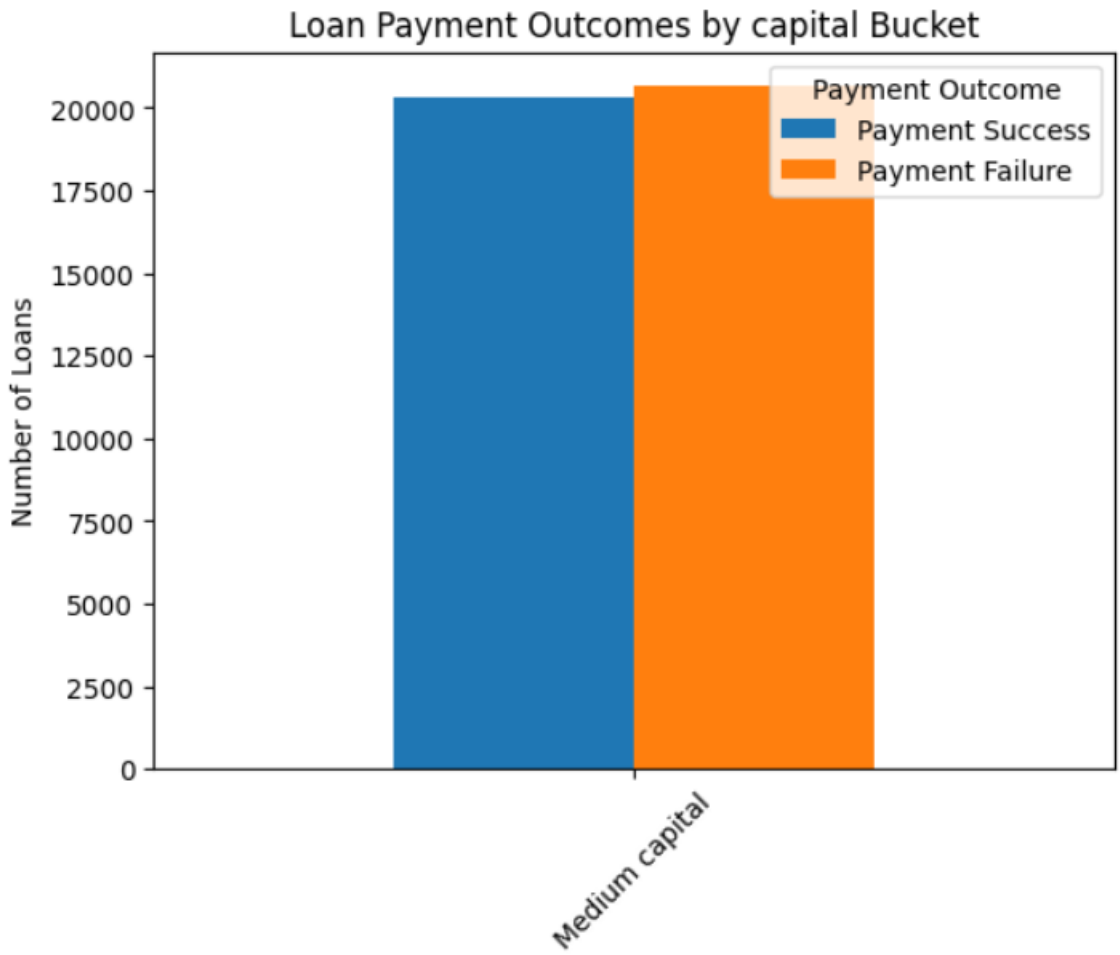

Figure 6 Loan payment failure outcomes by bucket

An additional variable that could potentially induce bias is gender. The data revealed that the gender labelled 'P' had a disproportionate percentage of loan payment defaults when compared to other gender groups. This imbalance raises issues of potential unfair bias in predictive models when gender 'P' is used.

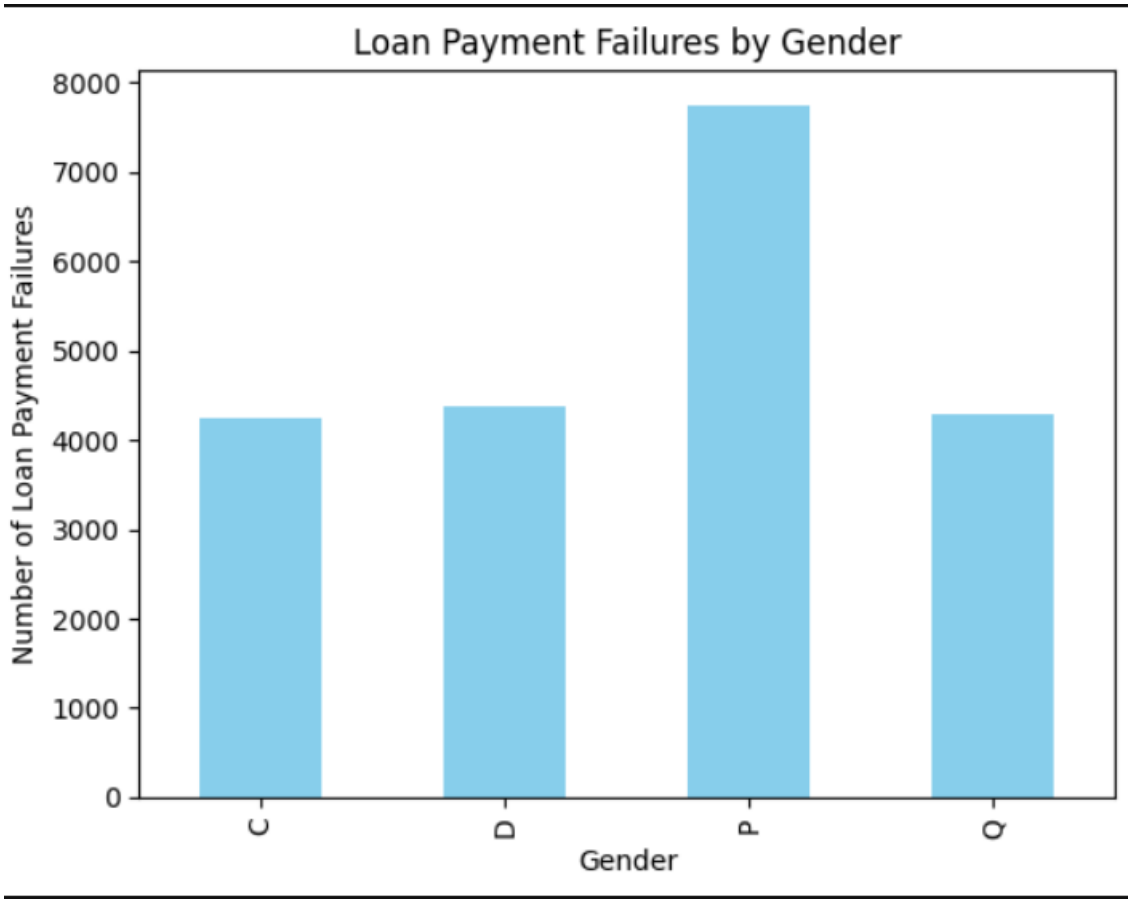

Figure 7 Loan payment Failures by Gender

A cross-sectional analysis of age groups and yearly incomes found that applicants in their forties with a medium annual income had the highest rate of loan payment failures. Based on the observed pattern of payment failures, the analysis suggests that the combination of these features together in an application may enhance the chance of bias.

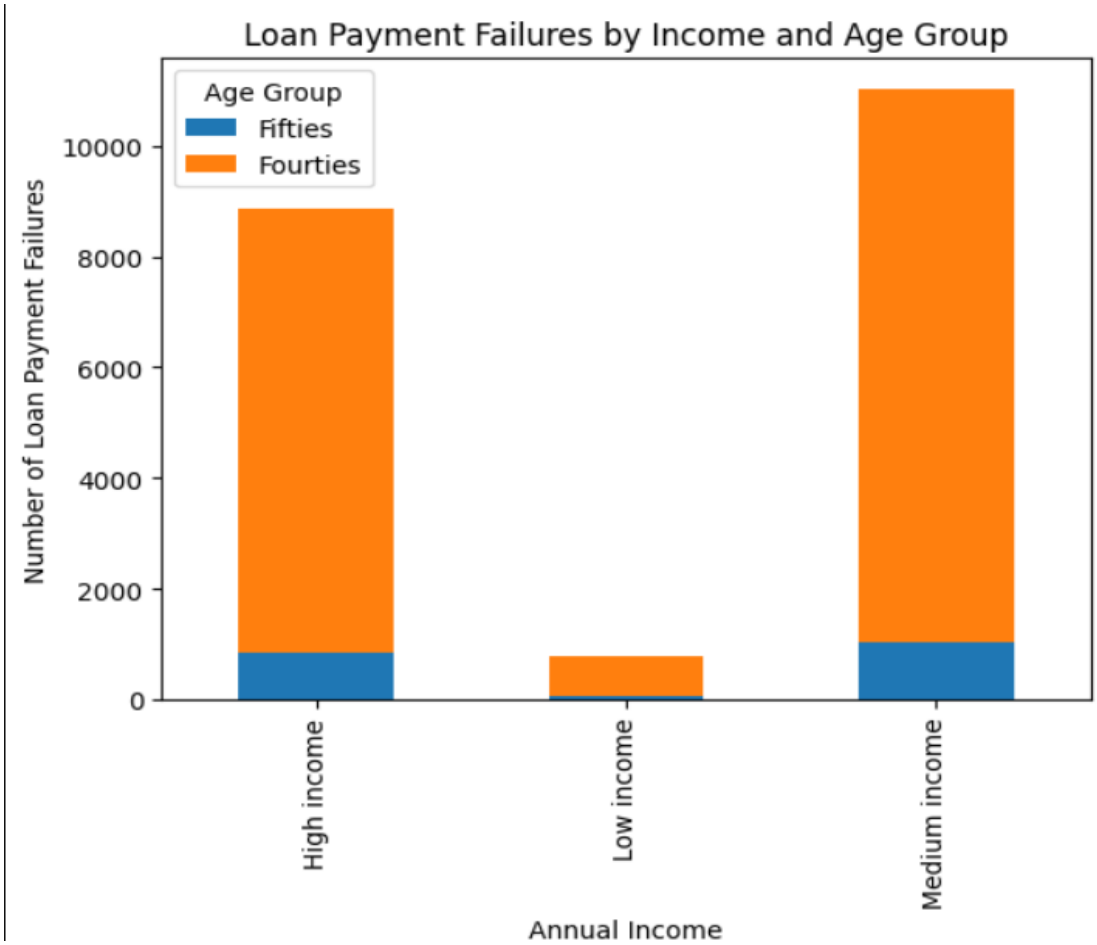

Figure 8 Loan payment failure by income and age group

By examining these patterns, it becomes clear that careful consideration must be given to these features to avoid perpetuating biases in predictive modelling.

Apriori Algorithm for Association Rule Minning

It is important to explore data bias and understand why and how the models are performing when given specific attributes. In this project the Apriori algorithm was employed with association rule mining to find any frequent item sets in the dataset that seem to be influencing the loan payment failure of an applicant. The algorithm was used only on the data which had the loan payment failure marked with a 1 as these were records that had failed to pay the loan. Harun et al (2017) defined apriori algorithm as a technique for mining frequent item sets that satisfies support and confidence by creating candidates during the joining and cutting phase. The algorithm was used on the encoded dataset with a minimum support parameter of 0.1. The Apriori algorithm only works on data that is encoded and does not work on data that has any other values that aren't encoded properly, e.g., feature containing the number 2 or strings. The most profound frequent items with a high support were the referee present feature and the gender feature, specifically the gender of P which had the highest support value out of all the genders.

These high support values can suggest that there Is bias when it comes to an applicant of a gender P and when it comes to having a reference present or not. Features like the age group, the covid years and capital seemed to appear often with other features suggesting that they might be possible bias when an application has one or more of these features in it.

Finally, association rules were created to discover any biased relationship between features and any true relationship that are within the data. The association rules aimed to find any frequent items occurring together more often in the dataset and by giving those combinations a high score of confidence, this is to indicate the strength of the pattern and possibly the impact on the loan payment failure outcome. The rules also included a lift score which measured the importance of the frequency in the dataset. If a lift score is one it is usually considered high but if it is greater than one, it might show that there's a likelihood of a meaningful link.

In the dataset the gender features, salary, years, income and age seemed to have been appearing more frequently together with other features. These features influenced the outcome of someone paying their loan.

```
Frequent Itemsets:
      support                                          itemsets
0    0.641578                                  (Refree_Present)
1    0.206002                                        (Gender_C)
2    0.211665                                        (Gender_D)
3    0.375121                                        (Gender_P)
4    0.207212                                        (Gender_Q)
..        ...                                               ...
650       0.1  (Scores_Medium score, Age_Group_Fourties, Refr...
651  0.117473  (Scores_Medium score, Age_Group_Fourties, Annu...
652  0.112633  (Year_Bucket_Low Years, Employment_type_Contra...
653  0.135963  (Year_Bucket_Covid Years, Age_Group_Fourties, ...
654  0.101016  (Year_Bucket_Low Years, Age_Group_Fourties, An...
```

Figure 9 Frequent items from apriori

K-Medoid Clustering

K-medoid clustering is a variant of K-means clustering and it was used on the data to partition it into clusters based on similarity and dissimilarity. Batra, A (2011) explained that, following a random selection of k medoids, the algorithm iteratively tries to make a better choice of medoids. A centrally situated object in a cluster can be used as a point of reference instead of the mean value of the points used by K-means. Similarly to the Apriori algorithm, the data used for K-medoid clustering was the ones which had a one in the loan payment failure column for better results. Four clusters were chosen for the medoid centroids, and a hamming distance was used to find the distance between data points. After the cluster was trained on the precomputed distance matrix, the data was clustered completely and four labels starting from zero to three were created and each feature point was assigned to its cluster. The apriori algorithm was then used again on the clustered data to reveal more patterns in the data, only large clusters were selected for this task.

The cluster sizes ranged from the zero-cluster having 8,187 data points, cluster one had 6,390, cluster two had 4,447 and the third cluster had 1,636 data points. The third cluster was dropped and the remaining clusters were used on the apriori algorithm, the frequent items after the clustering changed from 654 to 686. This revealed more patterns in the data than before. By using Principal component analysis to reduce the dimensions of the clustered data, it was easier to visualize the distribution of the clustered data. This visualization showed clusters in a two-dimensional space and clusters in groups and overlapping's were easily spotted.

```
Cluster ID
0    8187
1    6390
2    4447
3    1636
dtype: int64
C:\Users\salij\AppData\Local\Packages\PythonSoftwareFoundation.Pytho
ecationWarning: DataFrames with non-bool types result in worse compu
  warnings.warn(
Frequent Itemsets:
      support                                          itemsets
0    0.642241                                (Refree_Present)
1    0.205214                                       (Gender_C)
2    0.211312                                       (Gender_D)
3    0.377155                                       (Gender_P)
4    0.206318                                       (Gender_Q)
..        ...                                              ...
682  0.100399  (Scores_Medium score, Age_Group_Fourties, Refr...
683  0.120269  (Scores_Medium score, Age_Group_Fourties, Annu...
684  0.121058  (Year_Bucket_Low Years, Employment_type_Contra...
685  0.120847  (Year_Bucket_Covid Years, Age_Group_Fourties, ...
686  0.101714  (Year_Bucket_Low Years, Age_Group_Fourties, An...
```

Figure 10 Large clustered frequent items

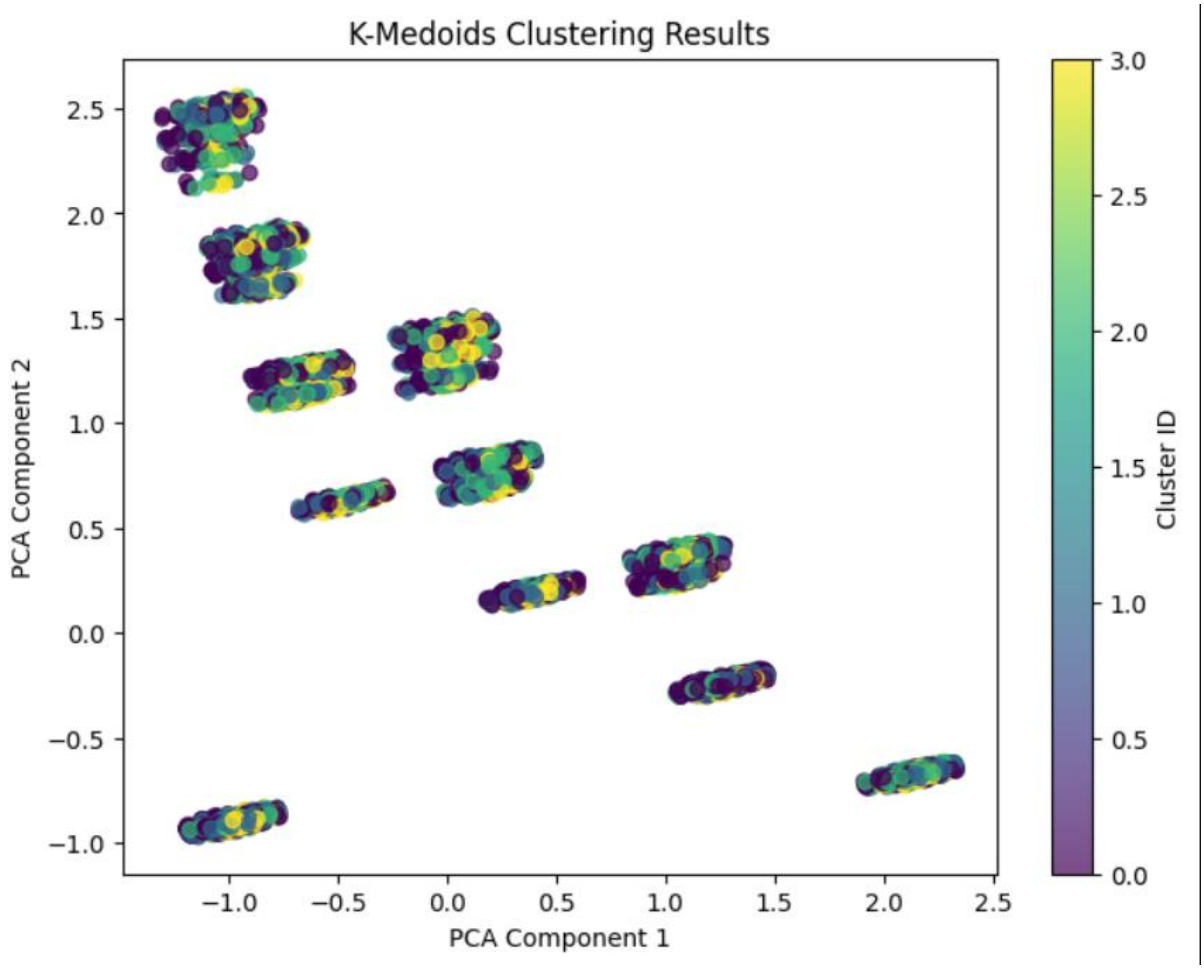

Figure 11 PCA component visualisation

Learning the Classification Models

The data was trained on several classification algorithms to compare each of the classifier's performance on the data. The classification algorithms used on the data were a Decision tree, Naïve Bayes, Random tree and an artificial neural network. These classifiers were chosen for their ability to manage large scale data efficiently.

Decision Tree Classifier

Chauhan, N.K. and Singh, K., (2022) defined a decision tree as a supervised learning tree-like structure in which each node stands for an attribute value within an instance to detect for a class and each branch is a value adopted by a node. The model was trained on the one hot encoded data, the data was split into X data which held the whole dataset. The Y data had the labels which was the loan payment failure feature. The data was then split into a twenty percent for testing and eighty percent for training by using the train test split function from sklearn.

For the model to perform well, hyperparameters needed to be selected carefully and randomly choosing these parameters can affect its performance in general. The model's performance was evaluated by using cross validation and f1 score. A parameter grid was used on the model with parameters of criterion to Gini and entropy, max depth of 3 up to 6, a random state of forty-two and minimum samples split of two up to 10. A grid search object was then created with the decision tree, the parameter grid and cross validation of 5 k-fold as parameters, the grid search object was then trained on the on the entire dataset to find the best parameters for the decision tree model. The best parameters found after training the grid search were then applied to the decision tree, these parameters were as follows, criterion to be entropy, maximum depth of six minimum sample splits of 5 and random state of 42. Finally, the model was then evaluated using cross validation scores and archived an accuracy of 71% after being evaluated on the unseen data. The test error on the on the on the 20% data was 28.707%.

Naïve Bayes

The Naïve bayes classifier is also a supervised classification model based on a conditional probabilistic technique that use the Bayes theorem to detect all compounds contained in the dataset, Chauhan, N.K. and Singh, K (2022). Cross validation and grid search was also used on the model to ensure exceptional performance and well evaluated accuracy scores, the data used on the model was split into a 20% for testing and 80% for training. A parameter grid for the model was created with variable smoothing as the only hyperparameter in the grid, the data was then trained on the grid search with cross validation using K fold of 5. After this the best parameters were then used on the model, the best parameter selected was variable smoothing of "1e-06". The model managed to archive an accuracy of 56.402% after being trained and evaluated on the unseen data. The test error on the 20% test data was 43.597%

Random Tree Classifier

Random forests are a method devised by Leo Breiman in the 2000s for constructing a prediction ensemble using a group of decision trees that grow in randomly chosen data subspaces, Biau, G., (2012). Cross validation and grid search was used on this model as well for better model validation and performance. The parameters used in the grid were, number of estimators which was chosen to be 100, max depth of 10, minimum sample split of 10, minimum sample leaf of 4 and random state of 42. The model was the trained using these parameters and the 80% training, the accuracy of the model was then confirmed by cross validation of k-fold of 5. The models accurately estimated to be 71.129% similarly to the decision tree classifier's accuracy. The test error on the 20% test data was 28.317%.

Support Vector Machine

Vapnik pioneered the SVM approach to dealing with classification and regression models. SVM is a supervised discriminative linear approach to binary classification using an explicit hyperplane, Chauhan, N.K. and Singh, K (2022). The model was trained on scaled data to ensure it is not biased towards certain features in the data and to improve the efficiency of the training process. The model was trained using parameters found by using the grid search method and the best parameters found were, C of 1, kernel of rbf and gamma of scale. These parameters were then used on the model and managed to achieve 71.182% accuracy almost closer to the decision tree and the random forest tree model. The test error on the 20-test data using the best parameters was 28.170%.

Multiple Layer Perceptron Artificial Neaural Network

MLP is a neural network-based efficient and stable method for solving nonlinear and difficult classification problems. It is made up of many neurons that are organized into input, hidden, and output layers, Chauhan, N.K. and Singh, K (2022). The model was also trained on the scaled data as this model is sensitive to the scale of inputs and using scaling data can help prevent any bias and efficiently improve training time. Cross validation and grid search was used on this model just like the others with the parameter grid having multiple hyperparameters. These parameters were chosen by the grid search, and they were as follows, hidden layers from set to 25, activation functions of Relu, Adam solver, alpha of 0.01 and a random state of 42. The model managed to get an accuracy of 68.963%. The test error on the 20% data after using best parameters was 31.036%.

To assess the model's performance during the training and validation phases, a learning curve was created. The graph illustrates the increase in model accuracy as training data is added incrementally. It also emphasizes the improvement in cross-validation accuracy, which supplies insights into the model's generalizability and effectiveness in learning from fresh data. This image is useful for understanding the relationship between training size and model proficiency, and it may be used to perfect the training process for better predictive outputs.

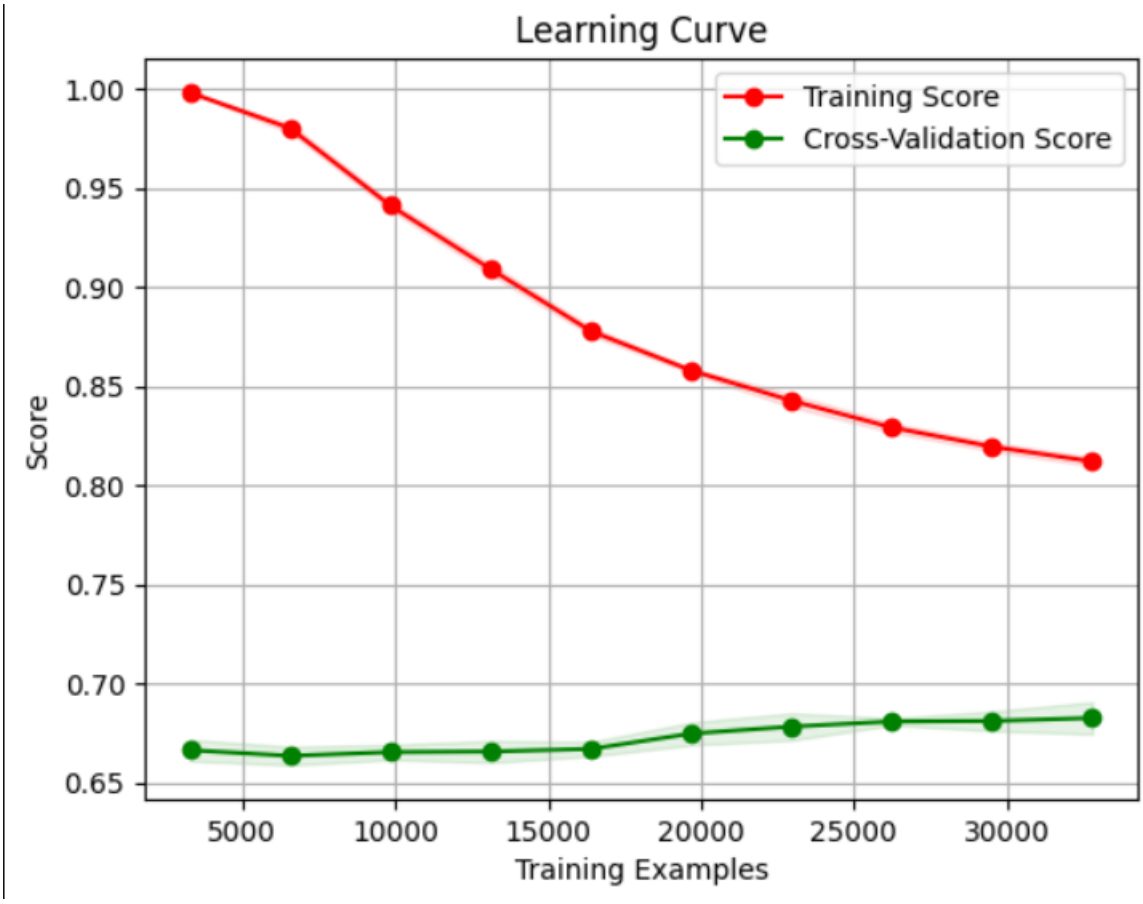

Figure 12 Learning Curve for Artificial Neural Network

Bias test Results

The best model out of the five models was the support vector machine with an accuracy of 71.182% and a test error of 28.170%. The features the model was trained on were evaluated for bias through the first data exploration process and it was discovered that these features had an impact on the classifier when making predictions. The bias on the classifier was assessed by using feature permutation importance, this method gave each feature a score based on how important it is. This importance can show how much the features are affecting the outcome of the classifier and causing some unethical bias.

	Feature	Importance
0	number_contractual_jobs_in_refs	0.053463
1	Year_Bucket_Covid Years	0.050293
2	low_salary	0.037488
3	Proposed_Business_Sector_Civil	0.033976
4	high_salary	0.015976
5	Annual_Income_Low income	0.015585
6	Gender_P	0.012854
7	medium_salary	0.010171
8	Refree_Present	0.007707
9	Employment_type_Self Business	0.007634
10	number_salaried	0.005098
11	Employment_type_Contractual	0.004951
12	Year_Bucket_Low Years	0.003756
13	Address_XYZ	0.003537
14	Gender_Q	0.003049
15	Education_STEM	0.002439
16	Proposed_Business_Sector_IT	0.002366
17	Annual_Income_Medium income	0.001829
18	Current_Employment_Sector_Agriculture	0.001805
19	number_selfbusiness_jobs	0.001805
20	Address_GHI	0.001780
21	Employment_type_Salaried	0.001756
22	Address_MNP	0.001756
23	Annual_Income_High income	0.001610
24	Age_Group_Fourties	0.001463
25	Age_Group_Fifties	0.001463

Figure 13 permutation feature importance

Reflection

Recognizing potential bias in features is an important component of responsible AI techniques. It indicates an understanding of how model inputs might influence outcomes, which is critical for maintaining fairness and avoiding discrimination. The application of feature permutation importance is a proactive move toward more openness. It is possible to determine which attributes may be contributing to biased outcomes by quantifying the influence of each feature on the model's predictions.

Conclusion

Data preprocessing is essential for preparing raw data for analysis. This entails cleaning and converting the data, which helps future analysis and modelling. Feature engineering refines the dataset even further, allowing for the extraction of useful insights that improve model performance. When using the Apriori algorithm for data exploration, it is possible to find item correlations and patterns that may introduce bias into the model. This is supplemented by K-medoid clustering, which pinpoints clusters within the loan payment failure data, supplying a deeper understanding of the associations that may contribute to loan payment failures. The application of various machine learning algorithms, fine-tuned and rigorously evaluated through grid searches and cross-validation, ensures a thorough assessment of model performance. Among the classifiers evaluated, the support vector machine appeared as the most effective, demonstrating superior performance on the dataset.

APENDIX

```python
#Filling all NaN in refs
data['Refree 1'].fillna('Not available', inplace=True)
data['Refree 2'].fillna('Not available', inplace=True)
data['Refree 3'].fillna('Not available', inplace=True)
```

```python
#Descritinizing Age
data['Age at which Loan was Taken'].astype(str)
for i, row in data.iterrows():
    age = row['Age at which Loan was Taken']

    if int(age) >= 40 and int(age) < 50:
        data.at[i, 'Age Group'] = 'Fourties'
    elif int(age) >= 50:
        data.at[i, 'Age Group'] = 'Fifties'
```

```python
#BINNING FOR YEARS
cut_points = [1998, 2010, 2019, 2021, 2022]
# DefinING the labels for the bins
labels = ['Low Years', 'Median Years', 'Covid Years', 'High Years']
# Bin the years
data['Year Bucket'] = pd.cut(data['Year of Application'], bins=cut_points, labels=labels, include_lowest=True)

# Print the year bins
data
```

```python
#Feature engineering for contractual jobs in refs
data['number_contractual_jobs_in_refs'] = data.apply(lambda row: sum('Contractual' in refree for refree in row[['Refree 1', 'Refree 2', 'Refree 3']]), axis=1)
# Convert the new column to numeric type
data['number_contractual_jobs_in_refs'] = pd.to_numeric(data['number_contractual_jobs_in_refs'], errors='coerce')


#Feature engineering for self business jobs in refs
data['number_selfbusiness_jobs'] = data.apply(lambda row: sum('Self Business' in refree for refree in row[['Refree 1', 'Refree 2', 'Refree 3']]), axis=1)

# Convert the new column to numeric type
data['number_selfbusiness_jobs'] = pd.to_numeric(data['number_selfbusiness_jobs'], errors='coerce')

#Feature engineering for salaried jobs in refs
data['number_salaried'] = data.apply(lambda row: sum('Salaried' in refree for refree in row[['Refree 1', 'Refree 2', 'Refree 3']]), axis=1)

# Convert the new column to numeric type
data['number_salaried'] = pd.to_numeric(data['number_salaried'], errors='coerce')
```

```python
# Extracting salarie amount from the tuple
salaries = data[['Refree 1', 'Refree 2', 'Refree 3']].apply(lambda row: [int(val.split(':')[1]) for val in row if 'Salaried' in val], axis=1)

# Categorize salaries into buckets
data['high_salary'] = salaries.apply(lambda salary_list: sum(salary >= 90000 for salary in salary_list))
data['medium_salary'] = salaries.apply(lambda salary_list: sum(50000 <= salary <= 89999 for salary in salary_list))
data['low_salary'] = salaries.apply(lambda salary_list: sum(salary <= 49999 for salary in salary_list))
```

```python
#ONE HOT ENCODING
encoded_data = pd.get_dummies(new_data)
encoded_data
```

```python
#NORMALISATION USING STANDARD SCALER
scaler = StandardScaler()
scaler.fit(encoded_data)
norm_data_scaled = pd.DataFrame(scaler.transform(encoded_data))
encoded_data
```

```python
# Apply Apriori algorithm to find frequent itemsets
apri_data = encoded_data.drop(['number_contractual_jobs_in_refs','number_selfbusiness_jobs','number_salaried','high_salary','medium_salary','low_salary'], axis=1)
payment_1 = apri_data[apri_data['Loan_Payment_Failure'] == 1]
payment_drop = payment_1.drop(['Loan_Payment_Failure'], axis=1)
frequent_itemsets = apriori(payment_drop,  use_colnames=True, min_support=0.1)

# Display the frequent itemsets and association rules
print("Frequent Itemsets:")
print(frequent_itemsets)
```

```python
frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))
rules = association_rules(frequent_itemsets, metric="confidence" )#min_threshol1005
#pd.set_option('display.max_columns', None)
#rules.head(50)
rules
```

```python
#k-MEDOID CLUSTERS

payment_1_km = encoded_data[encoded_data['Loan_Payment_Failure'] == 1]
km_data = payment_1_km.reset_index(drop=True)

# precompute the distances using cosine metric
distance_matrix = pairwise_distances(km_data, metric='euclidean')
#np.isnan(distance_matrix)
#np.count_nonzero(np.isnan(distance_matrix))

modelkm = KMedoids(n_clusters=4, metric='precomputed', random_state=0)
km = modelkm.fit(distance_matrix)
print(km.labels_)
print(km.inertia_)
cluslabels = pd.DataFrame(km.labels_,columns=['Cluster ID'])
result = pd.concat((km_data,cluslabels), axis=1)
result
```

```python
#K-MEDOID LARGE CLUSTER WITH APRIORI

cluster_sizes = result.groupby('Cluster ID').size()
#print(cluster_sizes)
large_cluster_threshold = 3000
large_clusters = cluster_sizes[cluster_sizes > large_cluster_threshold]
print(large_clusters)
large_cluster_ids = large_clusters.index

large_cluster_data = result[result['Cluster ID'].isin(large_cluster_ids)]

aprik_data = large_cluster_data.drop(['number_contractual_jobs_in_refs','number_selfbusiness_jobs','number_salaried','high_salary','medium_salary','low_salary', 'Cluster ID'], axis=1)

paymentk_1 = aprik_data[aprik_data['Loan_Payment_Failure'] == 1]
paymentk_drop = paymentk_1.drop(['Loan_Payment_Failure'], axis=1)
frequentk_itemsets = apriori(paymentk_drop,  use_colnames=True, min_support=0.1)#

# # Display the frequent itemsets and association rules
print("Frequent Itemsets:")
print(frequentk_itemsets)

frequentk_itemsets['length'] = frequentk_itemsets['itemsets'].apply(lambda x: len(x))
rules = association_rules(frequentk_itemsets, metric="confidence" )#min_threshold=0.5
rules
```

```python
#DECISION TREE CLASSIFIER with a Grid search a Cross validation of k-fold 5
from sklearn.metrics import f1_score

X = encoded_data.drop(['Loan_Payment_Failure'], axis=1)
y = encoded_data.iloc[:, 1]



X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

#parameter grid for grid search
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 4, 5, 6],
    'random_state': [42],
    'min_samples_split':[2,5,10]
}

model = DecisionTreeClassifier()

#grid search object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5)

#Fitting grid search to the training data
#grid_search.fit(X_train, y_train)
grid_search.fit(X, y)
best_params = grid_search.best_params_

dt_clf = DecisionTreeClassifier(**best_params)
dt_clf.fit(X_train, y_train)
dt_pred = dt_clf.predict(X_test)
```

```python
#NAIVE BAYES

param_grid = {
    'var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6]
}

nb_clf = GaussianNB()

# cross-validation on the model and grid search
grid_search = GridSearchCV(nb_clf, param_grid, cv=5)
grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_
print("Best Parameters:", best_params)

nb_clf = GaussianNB(**best_params)
nb_clf.fit(X_train, y_train)

# Make predictions on the test data
Nb_pred = nb_clf.predict(X_test)

# Calculate the accuracy score
accuracy = accuracy_score(y_test, Nb_pred)

print("Cross-Validation Scores:", cv_scores)
print("Accuracy Score:", accuracy)
print(confusion_matrix(y_test, Nb_pred), ": is the confusion matrix")
```

```python
# RANDOM FOREST CLASSIFIER

param_grid = {
    'n_estimators': [20, 50, 100],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'random_state': [42]
}


rf_clf = RandomForestClassifier()
grid_search = GridSearchCV(rf_clf, param_grid, cv=5)

#grid_search.fit(X_train, y_train)
grid_search.fit(X, y)
# Get the best parameters
best_params = grid_search.best_params_

rf_clf_best = RandomForestClassifier(**best_params)

#cross-validation on the training data
cv_scores = cross_val_score(rf_clf_best, X_train, y_train, cv=5)

rf_clf_best.fit(X_train, y_train)

y_pred = rf_clf_best.predict(X_test)

# Calculating the accuracy score
accuracy = accuracy_score(y_test, y_pred)

print("Best Parameters:", best_params)
print("Cross-Validation Scores:", cv_scores)
print("Accuracy Score:", accuracy)
print(confusion_matrix(y_test, y_pred), ": is the confusion matrix")
```

```python
#SVM TRAINED WITH SCALED DATA
X = norm_data_scaled.drop(['Loan_Payment_Failure'], axis=1)
y = norm_data_scaled['Loan_Payment_Failure'].astype('category').cat.codes
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

svm_clf = SVC()
grid_search = GridSearchCV(svm_clf, param_grid, cv=5)
#grid_search.fit(X_train, y_train)
grid_search.fit(X, y)

best_params = grid_search.best_params_
svm_clf_best = SVC(**best_params)

cv_scores = cross_val_score(svm_clf_best, X_train, y_train, cv=5)
svm_clf_best.fit(X_train, y_train)
y_pred = svm_clf_best.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Best Parameters:", best_params)
print("Cross-Validation Scores:", cv_scores)
print("Accuracy Score:", accuracy)
```

```python
#ARTIFIIAL NEUARL NETWORK WITH SCALED DATA
X = norm_data_scaled.drop(['Loan_Payment_Failure'], axis=1)
y = norm_data_scaled['Loan_Payment_Failure'].astype('category').cat.codes
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

param_grid = {
    'hidden_layer_sizes': [(100,), (50, 50), (25, 25, 25)],
    'activation': ['relu', 'tanh'],
    'solver': ['adam'],
    'alpha': [0.0001, 0.001, 0.01],
    'random_state': [42]
}

ann_clf = MLPClassifier()
grid_search = GridSearchCV(ann_clf, param_grid, cv=5)
#grid_search.fit(X_train, y_train)
grid_search.fit(X, y)

best_params = grid_search.best_params_
ann_clf_best = MLPClassifier(**best_params)
cv_scores = cross_val_score(ann_clf_best, X_train, y_train, cv=5)
ann_clf_best.fit(X_train, y_train)
y_pred = ann_clf_best.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Best Parameters:", best_params)
print("Cross-Validation Scores:", cv_scores)
print("Accuracy Score:", accuracy)
```

```python
#BIAS TESTING ON ALL FEATURES BASED ON IMPORTANCE
from sklearn.inspection import permutation_importance
import pandas as pd

# Getting feature names
feature_names = X_train.columns

# Permutation importance
result = permutation_importance(svm_clf_best, X_test, y_test, scoring='accuracy')

# Getting importances
importances = result.importances_mean

# Creating  dataframe for feature imprtances
importances_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})

# Sorting features by importance
importances_df = importances_df.sort_values('Importance', ascending=False).reset_index(drop=True)

print(importances_df)
```

```python
#Learning curve
train_sizes, train_scores, test_scores = learning_curve(ann_clf_best, X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 10))

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
test_scores_mean = np.mean(test_scores, axis=1)
test_scores_std = np.std(test_scores, axis=1)

plt.figure()
plt.title("Learning Curve")
plt.xlabel("Training Examples")
plt.ylabel("Score")
plt.grid()

plt.fill_between(train_sizes, train_scores_mean - train_scores_std, train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, test_scores_mean - test_scores_std, test_scores_mean + test_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r", label="Training Score")
plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="Cross-Validation Score")

plt.legend(loc="best")
plt.show()
```

REFERENCES

*Gama, J., Torgo, L., & Soares, C. (1998). Dynamic discretization of continuous attributes. In Progress in Artificial Intelligence—IBERAMIA 98: 6th Ibero-American Conference on AI Lisbon, Portugal, October 5–9, 1998 Proceedings 6 (pp. 160-169). Springer Berlin Heidelberg.*Dynamic_Discretization_of_Continuous_Att20160907-13854-vrsbpt-libre.pdf (d1wqtxts1xzle7.cloudfront.net)

Nguyen, M. (2020) A Guide on Data Analysis. Bookdown. Available at: https://bookdown.org/mike/data_analysis/

Raschka, S., 2015. Python machine learning. Packt publishing ltd. Published:23 September 2015.(pp. 15) https://www.google.co.uk/books/edition/_/GOVOCwAAQBAJ?hl=en&gbpv=0

Harun, N.A., Makhtar, M., Abd Aziz, A., Zakaria, Z.A. and Syed, F., 2017. The application of apriori algorithm in predicting flood areas. management, 17, p.763. https://core.ac.uk/download/pdf/296921317.pdf

Batra, A., 2011, November. Analysis and approach: K-means and K-medoids data mining algorithms. In ICACCT, 5th IEEE International Conference on Advanced Computing & Communication Technologies (pp. 274-279). https://apiit.edu.in/downloads/all%20chapters/CHAPTER-59.pdf

Chauhan, N.K. and Singh, K., 2022. Performance assessment of machine learning classifiers using selective feature approaches for cervical cancer detection. Wireless Personal Communications, 124(3), pp.2335-2366. https://link.springer.com/article/10.1007/s11277-022-09467-7#citeas

Biau, G., 2012. Analysis of a random forests model. The Journal of Machine Learning Research, 13, pp.1063-1095. https://www.jmlr.org/papers/volume13/biau12a/biau12a.pdf